DATA NORMALIZATION & LLM EXTRACTION

Architecture, methodology and execution guide

Python Developer Intern Assignment

This document describes the implementation as documented in the project README. The solution processes retailer product JSON, extracts conditional promotional information with an LLM, validates the extracted values using Python, and produces a fixed-schema CSV.

Input	Processing	Output
Input.json 12 phone records	LLM extraction + deterministic Python validation	24 rows × 29 columns CSV + issues log

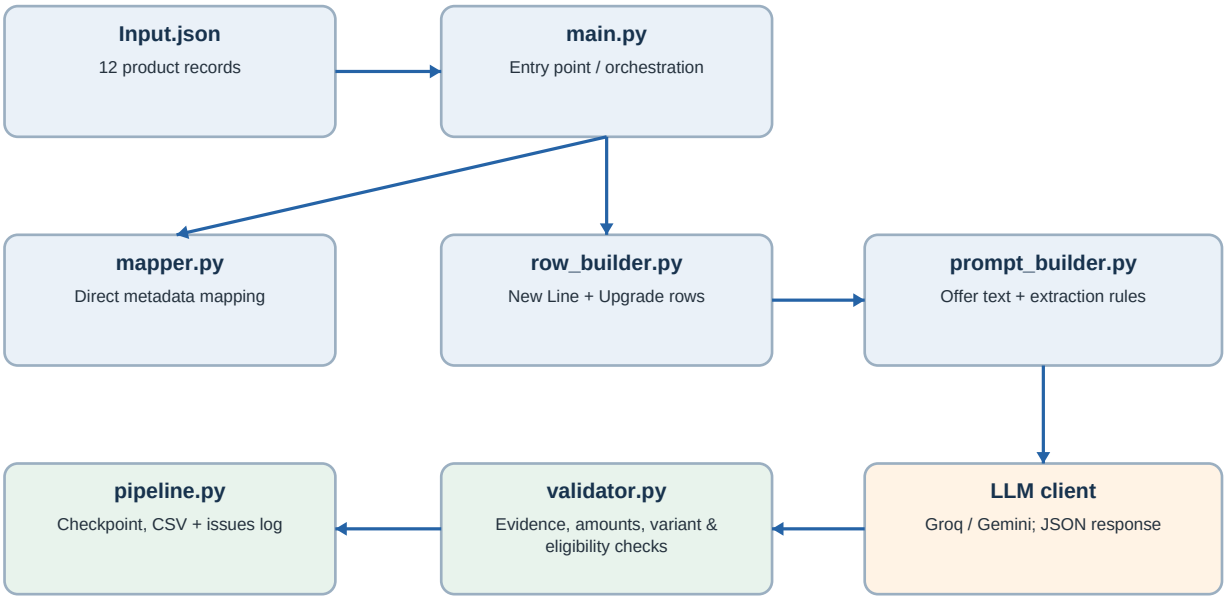
1. Objective and output

The input contains product metadata, pricing, offer text and accessory/add-on data. The main challenge is interpreting long promotional paragraphs whose eligibility may depend on storage variant or whether a customer is activating a New Line or upgrading an existing line.

The program creates exactly two output rows per product: one evaluated as **New Line** and one as **Upgrade**. For the documented input of 12 products, the expected output is 24 rows. The CSV uses the assignment's 29 fixed columns in the required order.

2. Architecture

PROCESSING ARCHITECTURE



Supporting modules: constants.py (schema), pwp.py (accessory offer calculation), env_loader.py, verify_output.py and tests.

The architecture separates direct field mapping, LLM-assisted interpretation, deterministic validation and output generation. The LLM proposes structured extractions; Python is responsible for accepting, rejecting, calculating and writing the final values.

3. Processing methodology

Step 1 — Read and map source data

The pipeline reads each product from Input.json. Basic fields such as product name, brand, variant, MSRP, stock, shipment and plan type are copied directly from the JSON through mapper.py; these fields do not require an LLM.

Step 2 — Create the two line-type scenarios

row_builder.py creates two rows for every product: New Line and Upgrade. Promotional eligibility is evaluated independently for each row, so an offer available to a new customer is not automatically copied to an upgrade row.

Step 3 — Ask the LLM to extract offer details

prompt_builder.py combines the relevant offer text with explicit extraction rules. The model is asked to return structured JSON. Each extracted amount must be accompanied by the exact source sentence supporting it. llm_client.py supports Groq or Gemini and handles retries, timeouts, JSON parsing and quota-related behavior.

Step 4 — Validate evidence and business rules in Python

validator.py checks the model response against the original source text. Values that cannot be proven from the text are dropped and left blank. This deterministic validation layer is central to the design: the LLM does not have final authority over numeric values or eligibility.

Validation rule	Implementation described in README
Evidence and exact amounts	The supporting quote must occur in the source, and the amount must match its exact digits. For example, $628 is rejected if the source says $628.36.
Variant matching	A device receives only the discount associated with its own storage size; a value listed for another variant is rejected.
Line-type eligibility	Switcher offers are excluded from Upgrade rows. Offers explicitly restricted to new lines are also excluded from Upgrade rows.
Finance calculation	Finance totals are calculated in Python: New Line = $200 + $240 + $10 = $450; Upgrade = $240 + $10 = $250.
Trade-in tier	The highest trade-in tier naming the relevant phone model is selected. A higher tier for a different model is not applied.
Non-duplication	A source sentence or monetary value is assigned to only one output category.

Step 5 — Accessory offer handling and output

pwp.py derives the documented PWP/GWP accessory offer from the accessories data without using the LLM. pipeline.py coordinates row processing, saves progress, checks the output and writes the CSV and issues log. The README reports that the AirPods 4 accessory saving is calculated as $30.00 and appears on both line types.

Step 6 — Verify the generated CSV

verify_output.py checks the final file against assignment rules and reports PASS, WARN or FAIL results. The documented run reported 21 passed checks, 0 warnings and 0 failures.

4. Project modules

Module / files	Responsibility
main.py	Entry point; reads input, runs processing and writes files.
constants.py	Defines the 29 output column names in the required order.
mapper.py	Maps basic product fields directly from the JSON.
row_builder.py	Builds New Line and Upgrade rows for each product.
prompt_builder.py	Builds the LLM prompt from extraction rules and offer text.
llm_client.py	Calls Groq or Gemini; handles retries, timeouts, parsing and quota handling.
validator.py	Validates LLM claims against source text and eligibility rules.
pwp.py	Calculates PWP/GWP accessory offers without an LLM.
pipeline.py	Coordinates processing, checkpointing, verification and output writing.
env_loader.py	Loads environment configuration from .env.
verify_output.py	Checks the generated CSV against assignment requirements.
check_connection.py	Sends a small request to test API key and model connectivity.
test_*.py / test_support.py	Test suite; README reports 187 tests.
Input.json / output/	Source product data and generated output files.

5. Reliability, error handling and test strategy

The LLM client includes retries and waiting for transient service errors, timeout handling, JSON parsing, quota handling and an optional fallback model for Gemini. Prompt size is limited by a configurable character budget; oversized trade-in text can be trimmed under the documented rules. The pipeline saves model responses in output/checkpoint.jsonl so a later run can continue without repeating completed requests.

The README documents three service issues encountered during development: Gemini HTTP 503 busy responses, quota HTTP 429 responses and Groq HTTP 413 prompt-size errors. It also notes that tests ignore the user's .env and environment configuration, and that real network calls inside tests fail intentionally. The test suite is run locally without an API key or internet connection.

6. Run instructions

Prerequisites

Python 3.9 or newer. The README says the project was tested with Python 3.12 and uses only the Python standard library; no package installation is required.

1. Clone the repository

```
git clone <the repo url>
cd <the repo folder>
```

2. Configure the provider

Create a .env file. For Groq, the README gives this example:

```
LLM_PROVIDER=groq
GROQ_API_KEY=your key here
GROQ_MODEL=qwen/qwen3.8-27b
GROQ_TPM_LIMIT=7000
```

For Gemini, set LLM_PROVIDER=gemini and provide GEMINI_API_KEY. The README states that .env is in .gitignore so credentials are not pushed to GitHub. Use the model and quota settings available to your own account.

3. Run tests (no API key required)

```
python -m unittest discover
```

The documented expected result is 187 tests passing.

4. Run a dry run

```
python main.py --dry-run
```

This creates output/dry_run_output.csv with the 24-row structure. Offer columns remain empty because the LLM is not called.

5. Run the full extraction

```
python main.py
```

The documented outputs are output/normalized_output.csv and output/issues.log. The process can be slow on free API quotas. If a daily limit is reached, it stops while retaining saved answers; rerun the same command later to continue.

6. Verify the result

```
python verify_output.py
```

The verifier reports PASS, WARN or FAIL for each check and prints a table of key columns.

7. Configuration and command options

Setting	Purpose
LLM_PROVIDER	groq, gemini or dryrun; README default: gemini.
GROQ_API_KEY / GROQ_MODEL	Groq credentials and model name.
GROQ_TPM_LIMIT	Tokens-per-minute limit used to pace requests.
PROMPT_CHAR_BUDGET	Maximum prompt size in characters; larger prompts are trimmed.
GEMINI_API_KEY / GEMINI_MODEL	Gemini credentials and model name.
GEMINI_FALLBACK_MODEL	Optional second Gemini model if the first is busy.
*_TIMEOUT / *_MAX_RETRIES	Timeout duration and retry count.

Option	Effect
--limit 1	Process only the first product (two rows).
--fresh	Discard saved answers and start again.
--dry-run	Run without an LLM call or API key.
--pause 5	Wait five seconds between calls.
--output file.csv	Write the CSV to a different filename.

8. Documented result

For the supplied input, the README reports 24 rows and 29 columns. It reports that all New Line rows have the $800 switcher and $200 service-line offer; seven products have a variant-matched unique discount; iPhone 17 and iPhone 18 Pro Max rows have model-matched maximum trade-in values of $930 and $1200 respectively; finance values are $450 for New Line and $250 for Upgrade; and the PWP value is $30.00 throughout.

9. Assumptions and known limitations

The assignment PDF defines all 29 column names but explains only some of their meanings. As described in the README, BASE, RESELLER LIST PRICE, Trade-In Value Type, FREE MEMORY UPGRADE, BOGO OFFER, SUBSCRIPTION BUNDLE and ADDITIONAL OFFERS are left blank where the input does not support a defensible value. The README also explains that its input does not contain the exact iPhone 17 Pro Max 256 GB example variant from the assignment; the implementation applies the same logic to the 1 TB and 2 TB variants present in its input.

The README identifies these limitations: the wording of switcher and finance descriptions is model-generated (the numeric amounts are checked); the solution has been tested only against this input file; and model providers/versions may differ across checkpointed responses. Free-tier quotas can make a full run slow.

10. Summary

The solution uses an LLM for interpretation of unstructured promotional language, but keeps critical correctness checks in deterministic Python code. Separating extraction from validation helps prevent unsupported values, variant mismatches, line-type eligibility errors and duplicate mapping before the normalized CSV is written.

Prepared from the project README supplied for this document. Implementation details and reported results are presented as described there; the source code and generated CSV were not independently audited for this PDF.